BSc (Hons) in **Information Technology Specialized in AI**

IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 02

> **Objective & Lecture Mapping:** In Lecture 03, we learned that a mathematically perfect "Table-Driven Agent" is impossible to build due to combinatorial explosion. Instead, we must write Agent Programs. Today, you will implement a **Simple Reflex Agent** using Condition-Action rules, observe its fatal flaws in a partially observable environment, and then upgrade it to a **Model-Based Agent** that maintains an internal memory state.

Part 1: Practical Implementation — Coding the Architectures

Step 1.1: Setting the Trap (Partial Observability)

> To see why Simple Reflex agents fail, we must handicap your agent's sensors. We are moving from a Fully Observable world to a Partially Observable one.

1. Open your `visual_grid_game.py` from Lab 01.
2. Locate the `get_percept(self)` method.

3. Modify it so it **no longer returns** the exact global coordinates (`agent_pos`). Instead, it should only return local booleans, e.g., `{'wall_ahead': True/False, 'food_here': True/False}` based on checking the adjacent cell in its current facing direction.

Step 1.2: The Simple Reflex Agent (Implementation & Failure)

1. Create a new class called `SimpleReflexAgent` .
2. Implement a `sense_and_act(self, percept)` method that uses strictly IF-THEN logic (Condition-Action rules). *Do not use an `__init__` method to store history.*
   Example Logic: `IF food_here THEN suck; IF wall_ahead THEN turn_left; ELSE move_forward`
3. Run the simulation. **Observe:** Watch the agent get trapped in a corner or a U-shaped wall, infinitely repeating a cycle (e.g., turn left, step forward, hit wall, turn left...).

Step 1.3: The Model-Based Agent (Memory & State)

1. Create a new class called `ModelBasedAgent` .
2. Implement an `__init__(self)` method to initialize an internal memory state (e.g., `self.visited_cells = set()` or a relative movement tracker).
3. In `sense_and_act(self, percept)` , first **update the state** (Transition & Sensor Model) by recording the current percept and the last action taken.
4. Update your IF-THEN rules to query the memory.
   Example Logic: `IF wall_ahead AND left_is_visited THEN turn_right`

5. Run the simulation. **Observe:** The agent should now remember where it has been, realize it is in a loop, and choose an alternate path to escape.

Part 2: Theoretical Evaluation & Lecture Mapping

> Based on your practical observations above, answer the following questions to map your code directly to the architectural theories covered in Lecture 02.

1. (Remember) According to Lecture 02, why is it impossible to program a mathematically perfect "Table-Driven Agent" for complex environments like Chess? What happens as the agent's lifetime increases?

A Table-Driven Agent needs one row for every possible percept sequence, not just one move — since decisions depend on the whole history so far. As the agent's lifetime increases, the length of the percept sequence grows, and the number of possible sequences explodes combinatorially — far more than there are atoms in the universe for a long enough game. f : P* → A, which is mathematically valid but impossible to store as an actual table. That's why we use an Agent Program instead.

2. (Understand) Look at the code you wrote for your `SimpleReflexAgent`. Identify and explain the specific lines of code that represent the "Condition-Action Rules" discussed in the lecture.

```
    return 'Up'
elif not percept['wall_right']:
    return 'Right'
```

Each if statement is a condition-action rule: the condition checks only the current percept (e.g. wall_up), and the action is what gets returned right after. Nothing looks at past percepts

3. (Analyze) Your `SimpleReflexAgent` likely got stuck in an infinite loop during Step 1.2. Based on the lecture, analyze exactly why this happened. How did the combination of "Partial Observability" and a lack of "Percept History" cause this

failure?

Two causes combined. First, partial observability — get_percept() only reveals the immediate adjacent cells, not the full map or agent position. Second, no percept history — the agent has no __init__, so it can't remember past percepts or actions. Once boxed in, it gets the exact same percept every step, and since its rule-mapping is fixed, the same percept always triggers the same action — causing it to loop Up→Down→Up→Down forever with no way to detect or escape the cycle.

4. (Evaluate) In Step 1.3, you added an internal state to your `ModelBasedAgent` . Evaluate how your specific code handles the "Transition Model" (how the world evolves) and the "Sensor Model" (how the agent's actions affect the world).

update_state() runs first inside sense_and_act(), before any decision is made. The Sensor Model part reads the incoming percept (wall_up/down/left/right, food_here) to see what the environment is showing right now. The Transition Model part uses self.last_action to predict how the world changed — updating self.position_estimate based on the assumption the move succeeded. This builds self.visited_cells, an internal map from the agent's own actions and percepts, letting it prefer unvisited cells and escape the loops that trap the SimpleReflexAgent.

Finalize and Lock Lab 02 Documentation

FACULTY OF COMPUTING